

Qualys Security Advisory

CVE-2015-3245 userhelper chfn() newline filtering

CVE-2015-3246 libuser passwd file handling

--[Summary]-----

The libuser library implements a standardized interface for manipulating and administering user and group accounts, and is installed by default on Linux distributions derived from Red Hat's codebase. During an internal code audit at Qualys, we discovered multiple libuser-related vulnerabilities that allow local users to perform denial-of-service and privilege-escalation attacks. As a proof of concept, we developed an unusual local root exploit against one of libuser's applications.

----[Vulnerability #1 (CVE-2015-3245 userhelper chfn() newline filtering)

We discovered a bug in userhelper, a setuid-root program from the usermode package that provides a basic interface to change a user's password, gecos information, and shell; its -f (Full Name), -o (Office), -p (Office Phone) and -h (Home Phone) command-line options are equivalent to those of the traditional chfn program.

userhelper's chfn() function verifies that the fields it was given on the command-line are sane (i.e., contain no forbidden characters). Unfortunately, these forbidden characters (":,=") do not include '\n' and allow local attackers to inject newline characters into /etc/passwd and alter this file in unexpected ways.

To the best of our knowledge, this bug is a local denial-of-service only: we were not able to turn it into a local root exploit, but maybe some creative minds will.

There is another, secondary aspect of this bug: userhelper depends on libuser to modify /etc/passwd, and libuser's format_generic() and generic_setpass() functions reject fields containing a ':' that would be interpreted as a field separator. Vulnerability #1 could have been prevented if libuser had also rejected '\n' characters.

----[Vulnerability #2 (CVE-2015-3246 libuser passwd file handling)

We discovered a bug in libuser itself: even though traditional programs like passwd, chfn, and chsh work on a temporary copy of /etc/passwd and eventually rename() it, libuser modifies /etc/passwd directly. Unfortunately, if anything goes wrong during these modifications, libuser may leave /etc/passwd in an inconsistent state.

This bug is not just another local denial-of-service: we were able to turn it into a local root exploit against userhelper and chfn (if linked with libuser).

There is also another, secondary aspect of this bug: glibc modules like nss and nscd do not expect /etc/passwd to be directly modified while they parse its contents, and programs from packages like shadow-utils and util-linux use lckpwwd() locks that are incompatible with libuser's fcntl() locks.

--[Exploitation Overview]-----

In this section, we outline our userhelper exploit against libuser's Vulnerability #2; later in this advisory, we explain how it can be easily adapted to chfn (if linked with libuser).

Our ultimate goal is to inject an arbitrary line into /etc/passwd (for example, the a-line "\na::0:0:::/\n") but we first need to understand how libuser's generic_mod() function modifies our own user's line in /etc/passwd:

- open() /etc/passwd for reading and writing (O_RDWR, but not O_APPEND nor O_TRUNC);
- acquire the file's fcntl() write-lock (an exclusive, but advisory lock);
- read() the file's contents (into a g_malloc()ated buffer);
- lseek() the file to the beginning of our user's line (and skip the unmodified lines that precede);
- write() our user's new, modified line (and the rest of the unmodified lines that follow) to the file;
- ftruncate() the file (if our user's new, modified line is shorter than the old one);
- release the file's fcntl() write-lock;
- close() the file.

Surprisingly, we only need two things in our toolbox in order to exploit this function and inject the a-line into /etc/passwd:

- a pencil and eraser that allows us to repeatedly write() and re-write() our own GECOS field (its length and last character in particular) in /etc/passwd: the userhelper program itself;
- a pair of scissors that allows us to interrupt write() with byte precision and avoid ftruncate(): the resource limit RLIMIT_FSIZE, "The maximum size of files that the process may create. Attempts to extend a file beyond this limit result in delivery of a SIGXFSZ signal. By default, this signal terminates a process, but a process can catch this signal instead, in which case the relevant system call (e.g., write(2), truncate(2)) fails with the error EFBIG."

For each character in the a-line (beginning with its last character and ending with its first character), we fork() a new process and execve() userhelper with:

- a GECOS field that allows us to write() the character to its target offset in /etc/passwd;
- an RLIMIT_FSIZE that allows us to terminate the process before it write()s or ftruncate()s the characters that follow.

In this example, the newline character '\n' is represented by |, and the last character written (before write() is interrupted by RLIMIT_FSIZE) is marked with ^:

```
...|...|user:x:1000:1000::/home/user:/bin/bash|...|...|
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAA:/home/user:/bin/bash|...|...|
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAA:/home/user:|^|...|...|
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAA:/home/|^|...|...|
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAA:/|^|...|...|
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAA:/:|^|...|...|
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAA::/^|...|...|
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAA0:/:|^|...|...|
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAA:0:/:|^|...|...|
```

```

...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA0:0::/:|...|...|
                                     ^
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA:0:0::/:|...|...|
                                     ^
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA:0:0::/:|...|...|
                                     ^
...|...|user:x:1000:1000:AAAAAAAAAAAAAAAAAAAAAAAAAAAAa:0:0::/:|...|...|
                                     ^
...|...|user:x:1000:1000:AAAAAAA:/home/user:/bin/bash|a:0:0::/:|...|...|
                                     ^
...|...|user:x:1000:1000::/home/user:/bin/bash|a:0:0::/:|...|...|

```

--[Exploitation Details]-----

In this section, we discuss the problems we encountered while developing our userhelper exploit, and how we solved them.

----[Problem #1 (missing fields)

At the end of our "Exploitation Overview" example, our home-directory and shell-program fields seem to magically reappear in /etc/passwd, although they were previously cut out by RLIMIT_FSIZE.

This magic trick introduces Problem #1: we cannot simply fork() a new process for each character in the a-line, execve() userhelper, and let it run until the character is written to its target offset in /etc/passwd, because libuser refuses to modify our user's line if some of its fields are missing.

In order to solve this Problem #1, we fork() a new process for each character in the a-line, execve() userhelper, and let it load our user's original, uncut line from /etc/passwd, but we SIGSTOP the process before it open()s /etc/passwd for writing. Only after we have started and stopped all userhelper processes can we safely SIGCONT them, one at a time.

----[Problem #2 (backup file)

Before libuser open()s /etc/passwd for writing, it creates a backup file named /etc/passwd- and if this backup fails, libuser refuses to modify /etc/passwd. Unfortunately, our RLIMIT_FSIZE also applies to the backup, which will fail if the RLIMIT_FSIZE is less than the size of /etc/passwd.

This introduces Problem #2: in apparent contradiction to what we just said, our exploit needs to decrease RLIMIT_FSIZE after each character it injects into /etc/passwd (as shown in the "Exploitation Overview" example).

In order to solve this Problem #2, we refine Problem #1's SIGSTOP/SIGCONT solution: we let each userhelper process load our user's original, uncut line from /etc/passwd, and SIGSTOP the process after it creates the backup file but before it modifies /etc/passwd. In other words, we have to win a race against generic_mod()'s system calls, which create the backup file and modify /etc/passwd:

- open() the passwd file /etc/passwd for reading;
- acquire the passwd file's fcntl() read-lock;
- open() the backup file /etc/passwd- for writing;
- acquire the backup file's fcntl() write-lock;
- read() from the passwd file;
- write() to the backup file;
- ftruncate() the backup file;

- release the backup file's fcntl() write-lock;
- close() the backup file;

- release the passwd file's fcntl() read-lock;
- close() the passwd file;

- open() /etc/passwd for reading and writing;
- [RACE WINDOW BEGINS]
- acquire the file's fcntl() write-lock: failure, sleep for a few microseconds;
- acquire the file's fcntl() write-lock: failure, sleep for a few microseconds;
- acquire the file's fcntl() write-lock: failure, sleep for a few microseconds;
- [RACE WINDOW ENDS]
- acquire the file's fcntl() write-lock: success;
- read() the file's contents;
- etc.

In order to reliably win this race against all userhelper processes (one for each character in the a-line), we:

- widen the race window. We acquire a read-lock on /etc/passwd before we execve() userhelper, which prevents libuser from acquiring the write-lock on /etc/passwd, and forces it to sleep for a few microseconds (LU_LOCK_TIMEOUT is 2, LU_MAX_LOCK_ATTEMPTS is 6).
- pinpoint the race window. We monitor the filesystem for the following sequence of inotify events:
 - . IN_CREATE on /etc if the backup file does not exist;
 - . IN_CLOSE_WRITE on the backup file;
 - . IN_CLOSE_NOWRITE on the passwd file;
 - . IN_OPEN on the passwd file.
- preempt the userhelper processes. We setpriority() them to the lowest priority, sched_setscheduler() them to SCHED_IDLE, and sched_setaffinity() them to the same CPU as our exploit.

----[Problem #3 (last user)

If our user's line is the last one in /etc/passwd, then the last character we inject into the file (the '\n' that ends our user's line and begins the a-line) is also the very last character of write()'s buffer, which introduces Problem #3: this last write() will not exceed our RLIMIT_FSIZE, and the consequent ftruncate() will delete the a-line from the end of /etc/passwd.

In order to solve this Problem #3:

- either we SIGKILL the last userhelper process after write() but before ftruncate(). We reliably win this race with an IN_MODIFY event on /etc/passwd and the "same CPU, different priorities" preemption of userhelper.
- or we exploit Vulnerability #1 and inject a '\n' into our own GECOS field. As far as libuser is concerned, this '\n' ends our user's line and begins a new one (with our leftover home-directory and shell-program fields): our user's line is no longer the last one in /etc/passwd.

----[Problem #4 (maximum GECOS_LENGTH)

As shown in our "Exploitation Overview" example, we only have two options for arbitrary character injection into /etc/passwd:

- either we use a character that we artificially inject through our own GECOS field (not an option for characters like ':' and '\n');
- or we reuse a character that is naturally present in /etc/passwd (our

only option for characters like ':' and '\n').

Unfortunately, both of these options might fail to inject a character after the end of /etc/passwd (a consequence of Problem #2):

- if our own GECOS field is too far away from the end of /etc/passwd (farther than userhelper's maximum GECOS_LENGTH, 127 characters);
- if the character is not already one of the last GECOS_LENGTH characters in /etc/passwd.

If faced with both of these problems, we solve the first one (and Problem #4) by repeatedly deleting lines from the end of /etc/passwd, until our own user's line is the last one in the file: we enlarge our own GECOS field, delete characters from the end of /etc/passwd with our RLIMIT_FSIZE scissors, shrink our GECOS field again, repeat.

----[Problem #5 (time complexity)

For each character in the a-line, we usually have to choose one of several (GECOS, RLIMIT_FSIZE) pairs that allow us to write the character to its target offset in /etc/passwd.

These pairs represent the nodes of a search tree that grows exponentially (with the number of characters in the a-line) but may contain few or no solutions. In order to avoid this tree's worst-case time complexity, we:

- inject the shortest a-line possible, "\na::0:0:::/\n";
- perform a recursive depth-first search on the tree, and return the first solution we find (instead of, for example, the solution that minimizes /etc/passwd's alterations);
- replace the a-line's username with a wildcard, and accept any lowercase character that is not already a username (the a-line's username was a major problem, because it is the last character we inject, and therefore occurs deep down the tree's branches; the a-line's '0' characters are only a minor problem, because they occur in the middle of the tree's branches, whence we can backtrack quickly).

----[chfn

util-linux's chfn from Red Hat's codebase is linked with libuser, and can be exploited by our public roothelper.c with just a few changes (left as an exercise for the interested reader):

- userhelper uses a simple Userhelper/Consolehelper request/response protocol in order to prompt for and read the user's password, but chfn uses traditional terminal interaction;
- if our user's line is the last one in /etc/passwd, we can exploit Vulnerability #1 against userhelper, but we have to win Problem #3's write/ftruncate race against chfn;
- userhelper returns 0/255 on success/failure, but chfn returns 0/1.

--[Acknowledgments]-----

We would like to thank Red Hat's Security Response Team and developers for promptly addressing these issues.